Madineni Music App

Full System Status, Architecture Flowcharts & Engineering Recommendations

Repository: github.com/madineni1144/Madineni-Music-	**Active Branch:** main (Up to date with origin/main)
Deployment Target: Ubuntu Server (/opt/musicapp/)	**Database:** PostgreSQL (music_app)
Core Ports & Runtime: Node.js Port 3000 | Airflow Port 8080	**Date of Report:** October 2026

1. Executive Summary & Status Overview

Madineni Music is a self-hosted audio streaming platform featuring a modern responsive frontend, a high-throughput Node.js Express API, a PostgreSQL metadata catalog, an automated Airflow ingestion pipeline integrating Google Drive, and a resilient synced-lyrics engine with LRCLIB fallback.

Git Connection Status: The local workspace is fully connected and tracked against GitHub repository madineni1144/Madineni-Music- on branch main. All tracked files are clean and synchronized without uncommitted changes.

2. System Architecture & Detailed Flowcharts

The application architecture consists of three core operational layers: **Client Experience**, **Backend Delivery**, and **Automated Ingestion**.

Diagram 2.1: End-to-End System Architecture

CLIENT LAYER	BACKEND API LAYER (Port 3000)	STORAGE & EXTERNAL SERVICES
• **Web UI:** frontend/index.html (AuraSound) • **Demo PWA:** frontend-demo/ (Obsidian Red) • **iOS App:** ios/ (Reserved native client) • **Audio Waveform:** WebAudio Visualizer	• **Express Gateway:** REST APIs (/api/songs, etc.) • **Static Streamers:** /music & /covers • **Lyrics Engine:** madineni-lyrics.cjs • **DB Connection Pool:** pg Pool (max 10)	• **PostgreSQL DB:** music_app • **Music Files:** /opt/musicapp/music/Songs • **Covers:** /opt/musicapp/covers • **Lyrics Provider:** lrclib.net (LRCLIB)
COMMUNICATION: HTTP / JSON	**COMMUNICATION: SQL / File I/O**	**AUTOMATION: Apache Airflow DAG**

Diagram 2.2: Automated Audio Ingestion Pipeline (Airflow DAG)

Scheduled to run 5 times daily (12 AM, 5 AM, 10 AM, 3 PM, 8 PM America/New_York) via madineni_audio_pipeline DAG.

Stage	Script Execution	Action & Transformation	Output / Destination
1. Ingestion	`download_incoming.py`	Downloads new tracks from Google Drive folder; creates import job record in PostgreSQL.	/opt/musicapp/staging/ PostgreSQL: import_jobs
2. Validation	`validate_audio.py`	Probes stream headers via ffprobe and executes decode test via ffmpeg to detect file corruption.	Validated MP3 streams or marked error items
3. Normalization	`normalize_metadata.py`	Reads ID3 tags; sanitizes Title, Artist, Album, Year, Track number; strips whitespace/illegal characters.	Normalized item metadata in PostgreSQL: import_items
4. Safe Storage	`store_audio.py`	Computes SHA-256 hash, organizes folder hierarchy, and safely stores sanitized MP3 files.	/opt/musicapp/music/Songs/ [Artist]/[Album]/[Track].mp3
5. Catalog Reg	`register_songs.py`	Upserts Artists and Albums; inserts track record in songs table with duration, format, and path.	PostgreSQL tables: artists, albums, songs

6. Verification	`playback_validate.py`	Performs synthetic stream checks against stored audio files to ensure browser compatibility.	Stream health verified
7. Archival	`archive_drive.py`	Moves processed files on Google Drive from incoming to archive folder to avoid duplicate runs.	Google Drive Archive
8. Reporting	`generate_report.py & upload_report.py`	Compiles Excel workbook and HTML report with metrics, warnings, and job throughput, and uploads to Drive.	Excel / HTML Report on Google Drive
9. Alerting	`send_email_summary.py`	Sends automated email summary to administrator via Gmail with status metrics and direct report links.	Admin Inbox Notification

Diagram 2.3: Lyrics Resolution & Fallback Engine

The lyrics subsystem (backend/madineni-lyrics.cjs) implements a 3-tier fallback architecture ensuring synchronized (.lrc) lyrics are available with minimal network latency and provider resilience.

Step / Priority	Resolution Mechanism	Behavior & Cache Policy
Tier 1: PostgreSQL Cache	Query table madineni_song_lyrics by song_id.	If lyrics exist and status is 'found', returns plain and synced LRC immediately (sub-millisecond latency). If 'not_found' record has not expired, returns 404 without querying provider.
Tier 2: Embedded ID3 Tags	Spawn read-embedded-lyrics.py using mutagen to inspect MP3 SYLT/USLT frames.	If audio file contains embedded synchronized lyrics (SYLT), extracts and converts milliseconds to standard LRC format [mm:ss.xx]; persists into database with source='embedded'.
Tier 3: Remote LRCLIB Provider	Fetch from https://lrclib.net with queued rate-limiter (max 12 concurrent requests).	Matches track title, artist, album, and duration. Automatically respects HTTP 429 / 503 Retry-After headers with cooldown backoff. Caches results into PostgreSQL.

3. Detailed Repository Component Inventory

Component / Area	Path / Primary Files	Stack	Key Responsibilities
Backend API	`backend/server.js`	Node.js, Express, pg	REST endpoints for songs, artists, albums, stats, playlists; static file streaming for music audio & album artwork.

Lyrics Subsystem	`backend/madineni-lyrics.cjs`
	`backend/lyrics-core.cjs`
	`backend/lyrics-player.js`
		Node.js CommonJS, Python Mutagen	Mounted Express lyrics router, LRCLIB API limiter, ID3 reader fallback, client-side floating synced lyrics player.

Web Frontend	`frontend/index.html` `frontend/aura-neon-music-app.html`	Vanilla HTML5, CSS3, JS	'AuraSound' dark neon audio player with reactive audio visualizer, dynamic equalizer knobs, playlist manager, and search.

Demo / PWA	`frontend-demo/index.html`
	`frontend-demo/index-real.html`
	`manifest.webmanifest`
		HTML5, PWA Web Manifest	'Obsidian Red' prototype player with full-screen album art layout, iOS/Android home screen icon support, and touch UI.

Audio Pipeline	`audio_pipeline/scripts/*.py` `audio_pipeline/requirements.txt`	Python 3, ffmpeg, ffprobe, psycopg2, google-api	End-to-end ingestion: downloading, media probing, ID3 normalization, disk storage, database registration, and reporting.

Airflow Orchestration	`airflow/dags/music_trending_import.py`
	`airflow/Dockerfile`
	`airflow/docker-compose.yaml`
		Apache Airflow 3.1.0, Docker Compose	Containerized job scheduling (5 runs/day), sequential task dependency execution, and error retry logic.

4. Express Backend API Endpoints

Method & Route	Parameters / Query	Response Structure	Description
`GET /api/health`	None	{success, status, database, time}	Liveness check & database ping.
`GET /api/songs`	?search, ?artist, ?album, ?limit, ?offset	Array of song objects with cover_art	Paginated song catalog with filtering.
`GET /api/songs/:id`	Path param: id (int)	{success: true, data: songObject}	Single song metadata details.
`GET /api/songs/:id/lyrics`	Path param: id (int)	{plainLyrics, syncedLyrics, source}	Synchronized lyrics from cache or LRCLIB.
`GET /api/artists`	None	Array of {id, name, song_count}	Alphabetical artist listing with track counts.
`GET /api/albums`	None	Array of {id, title, artist, cover_path}	Album catalog with artist references.
`GET /api/stats`	None	{songs, artists, albums, playlists}	Global library count statistics.
`POST /api/playlists`	Body: {name, description}	{success: true, data: newPlaylist}	Creates custom user playlist.
`GET /music/*` `GET /covers/*`	Static path	Binary audio/mpeg or image/jpeg stream	Direct audio playback & artwork streaming.

5. Prioritized Engineering Recommendations

Based on comprehensive static analysis of the codebase, the following prioritized improvements are recommended to increase reliability, security, portability, and throughput:

Priority 1: Security & Secrets Hardening (High Impact)

- **Remove Hardcoded Internal IPs:** Lines 1032–1035 of backend/server.js log private Wi-Fi and Tailscale IP addresses (192.168.4.204 and 100.83.79.17). These disclose internal subnet addressing. Replace with dynamic network interface lookups via os.networkInterfaces() or an environment variable.
- **Add Express Rate Limiting:** Endpoints like /api/songs and /api/songs/:id/lyrics currently have no rate limiter. Rapid client polling can exhaust the PostgreSQL connection pool (configured with max: 10 in server.js) or get the server's public IP blocked by LRCLIB. Implement express-rate-limit.
- **Payload Size Guards & CORS Restriction:** The server currently uses app.use(cors()) allowing unrestricted cross-origin requests. Restrict CORS to trusted origins, and add length constraints on user inputs in POST /api/playlists.
- **Add HTTP Security Headers:** Install helmet middleware to enable X-Content-Type-Options: nosniff and frame protection against clickjacking.

Priority 2: Environment Portability & Cross-Platform Support

- **Decouple Hardcoded Absolute Paths:** The server and ingestion scripts hardcode Linux paths such as /opt/musicapp/app/config/.env, /opt/musicapp/music/Songs, and /etc/musicapp.env. This prevents running or testing the app locally on Windows, macOS, or development containers. Introduce fallback environment variables (e.g. process.env.MUSIC_DIR || path.resolve(__dirname, '../music')).
- **Standardize .env.example:** Commit an .env.example file specifying required database variables (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD) and directory paths to streamline development setup.
- **Cross-Platform Python Executable:** In backend/madineni-lyrics.cjs line 133, the code invokes 'python3' directly via execFile. On Windows, the binary is typically 'python'. Use process.env.PYTHON_BIN || (process.platform === 'win32' ? 'python' : 'python3').

Priority 3: Docker & Airflow Image Synchronization

- **Synchronize Dockerfile with requirements.txt:** airflow/Dockerfile installs mutagen, pillow, and openpyxl, but misses all dependencies from audio_pipeline/requirements.txt (google-api-python-client, google-auth, psycopg2-binary, python-dotenv). If the Docker image is rebuilt, Airflow tasks will fail with ModuleNotFoundError. Update the Dockerfile to run pip install -r audio_pipeline/requirements.txt.

Priority 4: Database & Query Performance

- **Safe API Pagination Defaults:** In backend/server.js lines 157–162, when limit is missing, it defaults to 1,000,000. On large music libraries, an unpaginated query loads massive recordsets into Node.js heap memory and freezes the event loop. Lower default limit to 50 with an upper threshold of 500.
- **Trigram Indexing for ILIKE Searches:** The search query in /api/songs performs wildcard matching (title ILIKE %term%). Standard B-tree indexes cannot accelerate wildcard queries. Create PostgreSQL Trigram GIN indexes using pg_trgm on songs.title, artists.name, and albums.title.
- **Fix Inconsistent Single-Song Artwork:** GET /api/songs returns cover_art, but GET /api/songs/:id omits it from the SELECT query. Add the artwork path to the single-song query for consistent client rendering.
- **Track Database DDL in Git:** Add database/schema.sql and migration files so local developers or CI/CD pipelines can stand up identical databases from scratch.

Priority 5: Codebase Hygiene & Quick Cleanups

- **Remove Duplicate Script:** audio_pipeline/scripts/store_audio_v2.py is 100% byte-for-byte identical to store_audio.py. Retain only one canonical version to eliminate confusion.

- **Rename/Move frontend/madineni:** The file frontend/madineni is a Python file lacking an extension. Rename to .py and relocate to scripts/.
- **Browser Audio Cache Headers:** Add maxAge: '30d' to Express static music streaming (/music) so client browsers cache previously played songs, reducing repeated server bandwidth usage.